1 Step 2: Question-Level Behavioral Harmonization and Descriptive Analysis

1.1 Objective

The objective of Step 2 is to build a clean, question-level behavioral dataset that links:

- participant responses from the PsychoPy task logs;
- question metadata from `filtered_questions.json`;
- derived behavioral measures such as response time and correctness;
- descriptive comparisons between participant performance and the original ENEM item correctness.

This step remains intentionally simple. It focuses only on the behavioral side of the task and does not yet modify fNIRS files or analyze the fNIRS signals. Its purpose is to produce a well-organized and fully documented behavioral dataset that can later be aligned with the fNIRS recordings.

1.2 Main Goal

In practical terms, Step 2 should answer the following basic questions:

- Which questions were shown to each participant?
- Which answers did participants choose?
- Were those answers correct or incorrect?
- How long did participants take on each question?
- How do participant-level and item-level performance patterns differ between abstract and concrete questions?
- How does the observed participant accuracy compare with the original ENEM correctness for the same items?
- How do simple text properties of the items, such as word count and sentence length, relate to the observed behavioral patterns?

1.3 Raw Data Sources

Step 2 relies on two main raw sources.

1. PsychoPy task log

For each participant, the file

`enem_blocks_PIDxx_YYYYMMDD_HHMMSS.csv`

contains event-level task information, including at least the following columns:

```
t_abs
phase
block
trial_idx_in_block
question_number
question_year
question_type
question_field
marker_name
marker_code
rt_from_phase
choice
correct
button_click_time
option_view_time
note
```

The relevant phases include:

- `q_text_on`
- `q_stem_on`
- `q_options_on`
- `answer`
- `iti`
- `block_start`
- `block_end`

2. Question metadata

The file

`filtered_questions.json`

contains item-level metadata, including variables such as:

```
type
correctness
answer
total_word_count
sentence_count
sentence_length
```

Depending on the version of the file, it may also contain question text, field, year, options, and other metadata.

1.4 Interpretation of the Raw Structure

The PsychoPy task presents each item in stages:

1. text onset;
2. stem onset;
3. options onset;
4. answer.

This is useful because Step 2 can reconstruct not only a single response time, but also a small set of simple timing components:

- time from text onset to stem onset;
- time from stem onset to options onset;
- time from options onset to answer;
- total time from first presentation to answer.

This is still a basic descriptive approach, but it is more informative than keeping only one undifferentiated response-time measure.

1.5 Scope

This step includes:

- extraction of question-level behavioral data from the PsychoPy logs;
- harmonization of question identifiers;
- creation of a clean participant-question table;
- derivation of timing variables;
- derivation of participant correctness by comparing selected answers with the correct option;
- joining participant responses with question metadata from `filtered_questions.json`;

- descriptive summaries by question, question type, block, field, year, and participant;
- descriptive comparison between participant sample accuracy and ENEM correctness;
- export of CSV outputs, plots, and modular LaTeX files.

This step excludes:

- trigger repair in the fNIRS files;
- fNIRS preprocessing;
- fNIRS statistical analysis;
- advanced inferential modeling;
- mixed models, hierarchical models, or psychometric models.

1.6 Unit of Analysis

Step 2 should explicitly define three units of analysis.

Participant-question level

This is the main unit. Each row corresponds to one participant responding to one question.

Question level

Each row corresponds to one unique question, aggregated across all participants who saw that question.

Participant level

Each row corresponds to one participant, summarizing their overall task behavior and their behavior separately for abstract and concrete questions.

1.7 Core Principle

Because different participants may have seen different questions, and because block timing may cause some questions not to be reached, every summary in Step 2 must always report its denominator. In other words, all descriptive results must make clear:

- how many participants were exposed to the item;
- how many answered it;
- how many valid response times are available;
- how many valid correctness labels are available.

1.8 Recommended Question Identifier

A stable question identifier should be defined before any merge. A practical recommendation is:

```
question_id = question_year + "_" + question_field + "_" + question_number
```

If `filtered_questions.json` already contains a unique item ID, that identifier should be preferred. The same identifier must be used in all Step 2 outputs.

1.9 Derived Variables

The clean trial table should contain, at minimum, the following groups of variables.

Participant and session variables

- `participant_id`
- `timestamp`
- `source_log`

Task-position variables

- `block`
- `block_type` (Abstract or Concrete)
- `block_index_within_type` (1 to 5)
- `global_block_order` (1 to 10)
- `trial_idx_in_block` (1 to 3)
- `trial_idx_global`
- `order_condition` (AbstractFirst or ConcreteFirst)

Question metadata variables

- `question_id`
- `question_number`
- `question_year`
- `question_field`
- `question_type`
- `correct_answer`

- `enem_correctness`
- `total_word_count`
- `sentence_count`
- `sentence_length`

Behavioral response variables

- `chosen_answer`
- `participant_correct`
- `response_status`
- `answer_available`

Timing variables

- `t_text_on`
- `t_stem_on`
- `t_options_on`
- `t_answer`
- `rt_text_to_stem`
- `rt_stem_to_options`
- `rt_options_to_answer`
- `rt_text_to_answer`
- `rt_stem_to_answer`

Normalized timing variables

Because questions differ in length, it is useful to create very simple normalized timing measures:

- `sec_per_word` = `rt_text_to_answer / total_word_count`
- `sec_per_sentence` = `rt_text_to_answer / sentence_count`

These variables remain descriptive only. They are included because raw time alone may be misleading when items differ substantially in length.

1.10 Expected Outputs

Structured tables

```
01_question_file_manifest.csv
02_question_metadata_raw.csv
03_question_metadata_clean.csv
04_behavior_events_raw.csv
05_behavior_trial_raw.csv
06_behavior_trial_clean.csv
07_question_summary.csv
08_question_answer_distribution.csv
09_type_summary.csv
10_block_summary.csv
11_field_summary.csv
12_year_summary.csv
13_participant_behavior_summary.csv
14_exam_vs_sample_comparison.csv
15_behavior_missingness.csv
```

Plots

```
figures/step2/accuracy_by_type.png
figures/step2/rt_total_by_type.png
figures/step2/enem_vs_sample_accuracy.png
figures/step2/word_count_vs_rt.png
figures/step2/word_count_vs_accuracy.png
figures/step2/question_accuracy_comparison.png
figures/step2/question_rt_distribution.png
```

LaTeX outputs

```
reports/step2/questions/question_<question_id>.tex
reports/step2/groups/group_abstract_vs_concrete.tex
reports/step2/groups/group_blocks.tex
reports/step2/groups/group_fields.tex
reports/step2/step2_behavior_report.tex
```

1.11 Detailed Workflow

1.11.1 Step 2.1: Build the file manifest

Create a manifest that lists, for each participant:

- participant folder;
- PsychoPy log file name;

- timestamp extracted from file name;
- whether the log is readable;
- whether a paired demographic file exists;
- notes on any irregularity.

The purpose of this file is to ensure that all later behavioral analyses are traceable back to the raw source file.

1.11.2 Step 2.2: Parse and flatten `filtered_questions.json`

Convert `filtered_questions.json` into a tabular question-metadata file. For each question, extract at least:

- question ID;
- year;
- field;
- type;
- correct answer;
- ENEM correctness;
- total word count;
- sentence count;
- sentence length.

Save this as:

`02_question_metadata_raw.csv`

1.11.3 Step 2.3: Clean and validate question metadata

Create a cleaned question metadata table. At this stage:

- standardize field names;
- standardize type labels to exactly `Abstract` and `Concrete` or a fixed lower-case equivalent;
- validate that ENEM correctness is numeric and lies between 0 and 1;
- validate that correct answer is one of `A`, `B`, `C`, `D`, or `E`;
- validate that text-complexity variables are numeric and positive when present.

Save the cleaned table as:

`03_question_metadata_clean.csv`

1.11.4 Step 2.4: Extract event-level task data from the PsychoPy logs

Read all `enem_blocks` files and concatenate the relevant task rows into one raw event table. Keep only the fields needed for behavioral reconstruction:

- participant ID;
- timestamp;
- phase;
- block;
- trial index in block;
- question identifiers;
- `t_abs`;
- `choice`;
- `button_click_time`;
- `option_view_time`;
- marker fields;
- notes.

Save this as:

`04_behavior_events_raw.csv`

1.11.5 Step 2.5: Reconstruct one row per participant-question trial

Group the event table by participant, block, trial index, and question identifier, and reconstruct one trial row for each presented question.

For each trial, identify:

- the text onset event;
- the stem onset event;
- the options onset event;
- the answer event, if present.

This yields a raw trial table with one row per participant-question attempt. Save it as:

`05_behavior_trial_raw.csv`

1.11.6 Step 2.6: Derive timing variables

For each reconstructed trial, compute:

- `rt_text_to_stem = t_stem_on - t_text_on`
- `rt_stem_to_options = t_options_on - t_stem_on`
- `rt_options_to_answer = t_answer - t_options_on`
- `rt_text_to_answer = t_answer - t_text_on`
- `rt_stem_to_answer = t_answer - t_stem_on`

Also compute:

- `sec_per_word`
- `sec_per_sentence`

If any phase is missing, the derived timing variable should be set to missing and flagged explicitly.

1.11.7 Step 2.7: Derive answer and correctness variables

The PsychoPy log stores the participant's selected option in the `choice` column of the `answer` row. Use this selected option and compare it with the correct option from `filtered_questions.json`.

Create:

- `chosen_answer`
- `correct_answer`
- `participant_correct`

Recommended coding:

- `participant_correct = 1` if chosen answer matches correct answer;
- `participant_correct = 0` if chosen answer does not match correct answer;
- missing if the answer event is absent or unusable.

1.11.8 Step 2.8: Define response-status categories

Use simple explicit status labels:

- `ANSWERED_VALID`
- `ANSWERED_INVALID`

- `MISSING_ANSWER`
- `PARTIAL_TRIAL`
- `UNMATCHED_QUESTION`

These categories should be used throughout Step 2 to keep the data traceable and auditable.

1.11.9 Step 2.9: Join with question metadata

Merge the participant-question trial table with the cleaned question metadata. At this stage, validate:

- that the question type in the log matches the question type in the metadata;
- that the question year and field are consistent;
- that each presented item finds exactly one metadata row.

If any trial does not match cleanly to a question in the metadata, keep the row but flag it for manual review.

1.11.10 Step 2.10: Derive block-order variables

From the block labels such as `A1`, `A2`, `C1`, and so on, derive:

- `block_type`
- `block_index_within_type`
- `global_block_order`
- `order_condition`

This is important because the experimental script presents five blocks of one type first and then five blocks of the other type. Even in a basic descriptive workflow, order should be retained as a documented variable.

1.11.11 Step 2.11: Build the clean trial table

Save the final cleaned participant-question dataset as:

```
06_behavior_trial_clean.csv
```

This should be the main Step 2 analysis table. It should contain one row per participant-question trial, all relevant metadata, timing variables, correctness variables, and quality flags.

1.11.12 Step 2.12: Produce question-level summaries

For each unique question, compute:

- number of participants exposed;
- number of valid answers;
- number correct;
- sample accuracy;
- ENEM correctness;
- difference between sample accuracy and ENEM correctness;
- mean and median response time;
- standard deviation of response time;
- mean normalized timing measures;
- answer distribution across A–E.

Save these results as:

```
07_question_summary.csv
08_question_answer_distribution.csv
```

1.11.13 Step 2.13: Compare participant sample accuracy with ENEM correctness

For each question, define:

```
difficulty_gap = sample_accuracy - enem_correctness
```

This quantity should be interpreted descriptively:

- a positive value means the sample found the item easier than the original ENEM population;
- a negative value means the sample found the item harder than the original ENEM population.

This comparison is useful, but it must be described carefully. The two groups are not the same population, so this is a descriptive comparison, not a formal calibration exercise.

Save the comparison table as:

```
14_exam_vs_sample_comparison.csv
```

1.11.14 Step 2.14: Produce type-level summaries

Aggregate the clean trial table by question type:

- Abstract
- Concrete

For each type, compute:

- number of trials;
- number of valid answers;
- mean sample accuracy;
- mean response time;
- median response time;
- mean ENEM correctness of the presented items;
- mean word count;
- mean sentence count;
- mean sentence length.

Save this table as:

`09_type_summary.csv`

1.11.15 Step 2.15: Produce block-level summaries

Aggregate by:

- block label;
- block type;
- global block order;
- trial position within block.

For each grouping, compute:

- number of presented questions;
- number of answered questions;
- mean response time;
- mean accuracy;

- mean normalized timing measures.

This helps identify simple fatigue or order effects without introducing complex modeling.

Save the result as:

```
10_block_summary.csv
```

1.11.16 Step 2.16: Produce field-level and year-level summaries

Aggregate by:

- `question_field`
- `question_year`

For each group, compute:

- number of questions;
- number of valid trials;
- mean accuracy;
- mean ENEM correctness;
- mean response time.

Save these tables as:

```
11_field_summary.csv
12_year_summary.csv
```

1.11.17 Step 2.17: Produce participant-level summaries

For each participant, compute:

- number of questions shown;
- number of valid answers;
- overall accuracy;
- overall mean response time;
- abstract-question accuracy;
- concrete-question accuracy;
- abstract-question mean response time;
- concrete-question mean response time;

- order condition.

This participant-level table is especially useful for later merging with fNIRS summaries.
Save it as:

`13_participant_behavior_summary.csv`

1.11.18 Step 2.18: Produce missingness and quality tables

Create a missingness table that reports, for each variable and each response-status category:

- count;
- percentage.

Save it as:

`15_behavior_missingness.csv`

1.12 Minimal Descriptive Analyses to Include

A. Question-level analysis

For each question:

- sample size;
- sample accuracy;
- ENEM correctness;
- difficulty gap;
- answer distribution;
- mean and median response time;
- question length variables.

B. Abstract vs concrete analysis

Compare abstract and concrete questions descriptively on:

- sample accuracy;
- response time;
- normalized timing;
- average word count;

- average sentence count;
- average sentence length;
- average ENEM correctness.

C. Item-length analysis

Describe how item length relates to behavior using:

- scatter plot of word count versus mean response time;
- scatter plot of word count versus sample accuracy;
- optional tables sorted by longest questions and shortest questions.

D. Position analysis

Describe how behavior varies by:

- block order;
- question position within block;
- order condition.

1.13 Plots

The following plots are recommended for the basic Step 2 report.

Core plots

- bar plot of sample accuracy by question type;
- boxplot or violin-free boxplot of response time by question type;
- scatter plot of ENEM correctness versus sample accuracy;
- scatter plot of word count versus mean response time;
- scatter plot of word count versus sample accuracy.

Question-level plots

- bar plot of sample accuracy by question;
- paired comparison plot of sample accuracy and ENEM correctness by question;
- answer-distribution plot for each question.

Block and participant plots

- accuracy by global block order;
- response time by global block order;
- participant-level abstract versus concrete summary plot.

1.14 Per-Question LaTeX Reporting

One modular LaTeX file should be created for each question:

```
reports/step2/questions/question_<question_id>.tex
```

Each file should contain:

- question identifier;
- question metadata table;
- sample exposure count;
- answer distribution;
- sample accuracy;
- ENEM correctness;
- difficulty gap;
- response-time summary;
- corresponding plot or plots;
- notes on any data-quality issue.

1.15 Master LaTeX Report

Create:

```
reports/step2/step2_behavior_report.tex
```

This report should contain:

- a short introduction to the behavioral analysis;
- a description of the raw files and reconstruction logic;
- a section on item metadata;
- a section on overall task performance;

- a section on abstract versus concrete comparisons;
- a section on question-level summaries;
- a section on item-length summaries;
- a section on data-quality and missingness;
- `\input{...}` statements for modular question or group files.

1.16 Template for One Question Report

Each question-specific LaTeX file can follow a structure such as:

```
\subsection{Question <question_id>}
\label{sec:q_<question_id>}

\paragraph{Metadata.}
Year, field, type, correct answer, ENEM correctness, word count, sentence count, sentence leng

\paragraph{Exposure and validity.}
N exposed, N answered, N valid.

\paragraph{Performance.}
Sample accuracy, ENEM correctness, difficulty gap.

\paragraph{Timing.}
Mean, SD, median for total response time and normalized timing.

\paragraph{Answer distribution.}
Counts and percentages for A, B, C, D, and E.

\paragraph{Notes.}
Any issue with missing events, inconsistent metadata, or unusual answer patterns.
```

1.17 Quality-Control Checks

Before Step 2 is considered complete, the following checks should be run:

1. every participant log is readable;
2. each question trial has a valid participant ID and block label;
3. event phases are reconstructed correctly;
4. the answer event is identified correctly;
5. chosen answers are limited to A–E;
6. metadata merges uniquely to each question;

7. question type in the log matches question type in the metadata;
8. ENEM correctness values are within the interval $[0, 1]$;
9. response-time variables are non-negative;
10. normalized timing variables are computed only when denominators are valid;
11. denominators are reported in all summary tables.

1.18 What We Intentionally Do Not Do Yet

To keep Step 2 simple and robust, the following are intentionally postponed:

- no inferential tests between abstract and concrete items;
- no mixed-effects modeling;
- no item-response modeling;
- no formal equating between the sample and the ENEM population;
- no integration with fNIRS signals yet.

1.19 Minimal Completion Criteria for Step 2

Step 2 should be considered complete when:

- all participant PsychoPy logs have been read;
- a clean participant-question table exists;
- question metadata from `filtered_questions.json` have been merged;
- correctness and timing variables have been derived;
- item-level and type-level summary tables exist;
- sample accuracy has been descriptively compared with ENEM correctness;
- basic plots have been generated;
- modular LaTeX outputs have been generated;
- all data-quality issues have been documented.

1.20 Short Practical Summary

In simple terms, Step 2 should transform the PsychoPy task logs into one clean behavioral dataset with one row per participant-question response, enriched with item metadata from `filtered_questions.json`, and produce a basic descriptive analysis of:

- time;
- correctness;
- abstract versus concrete differences;
- question difficulty;
- comparison with original ENEM correctness;
- simple item-length effects.